

DSA Revision Notes

Striver's A2Z DSA Sheet

Batch 2 · Step 3: Arrays (Easy → Medium → Hard)

Problems	26 problems across Easy, Medium, and Hard
Patterns	Two Pointers · Prefix Sum · Hashing · Kadane's · Dutch Flag · Greedy
Source	takeuforward.org Raj Vikramaditya (Striver)
Format	Pattern · Intuition · Algorithm · Complexity · Traps · Trigger

Arrays are the most tested topic in coding interviews. Master every pattern here.

TABLE OF CONTENTS

EASY PROBLEMS (3.1)

- 3.1.1 Largest Element in an Array
- 3.1.2 Second Largest Element
- 3.1.3 Check if Array is Sorted
- 3.1.4 Remove Duplicates from Sorted Array
- 3.1.5 Left Rotate Array by One
- 3.1.6 Left Rotate Array by D Places
- 3.1.7 Move Zeros to End
- 3.1.8 Linear Search
- 3.1.9 Union and Intersection of Two Sorted Arrays
- 3.1.10 Find Missing Number
- 3.1.11 Maximum Consecutive Ones
- 3.1.12 Find the Single Number (XOR)
- 3.1.13 Longest Subarray with Sum K (Positives)
- 3.1.14 Longest Subarray with Sum K (Pos + Neg)

MEDIUM PROBLEMS (3.2)

- 3.2.1 Two Sum
- 3.2.2 Sort 0s 1s 2s — Dutch National Flag
- 3.2.3 Majority Element ($> N/2$) — Boyer-Moore Voting
- 3.2.4 Kadane's Algorithm — Maximum Subarray Sum
- 3.2.5 Print Subarray with Max Sum
- 3.2.6 Stock Buy and Sell
- 3.2.7 Rearrange Alternating Positive Negative
- 3.2.8 Next Permutation
- 3.2.9 Leaders in an Array
- 3.2.10 Longest Consecutive Sequence
- 3.2.11 Set Matrix Zeros
- 3.2.12 Rotate Matrix 90 Degrees
- 3.2.13 Spiral Order Matrix
- 3.2.14 Count Subarrays with Given Sum

HARD PROBLEMS (3.3)

- 3.3.1 Pascal's Triangle
- 3.3.2 Majority Element ($> N/3$) — Extended Voting
- 3.3.3 3-Sum Problem
- 3.3.4 4-Sum Problem
- 3.3.5 Largest Subarray with 0 Sum
- 3.3.6 Count Subarrays with XOR = K
- 3.3.7 Merge Overlapping Intervals
- 3.3.8 Merge Two Sorted Arrays Without Extra Space
- 3.3.9 Find Repeating and Missing Number
- 3.3.10 Count Inversions
- 3.3.11 Reverse Pairs
- 3.3.12 Maximum Product Subarray

STEP 3 — Arrays

3.1 EASY PROBLEMS

3.1.1 · Largest Element in an Array

TOPIC / PATTERN

Arrays — Linear Scan

CORE IDEA

Track the maximum value seen so far as you scan left to right in a single pass.

ALGORITHM

- 1. Initialize $\text{max} = \text{arr}[0]$.
- 2. For each element from index 1 to $N-1$: if $\text{arr}[i] > \text{max}$, update $\text{max} = \text{arr}[i]$.
- 3. Return max .

COMPLEXITY

Approach	Time	Space	Notes
Sorting	$O(N \log N)$	$O(1)$	Sort and return last — overkill
Linear Scan	$O(N)$	$O(1)$	Optimal — single pass

COMMON MISTAKES

- Initializing $\text{max} = 0$ — fails for all-negative arrays. Always init to $\text{arr}[0]$.

■ Interview Trigger

Single pass, track running max — $O(N)$.

3.1.2 · Second Largest Element (Without Sorting)

TOPIC / PATTERN

Arrays — Two-Variable Linear Scan

CORE IDEA

Maintain two variables: `largest` and `second_largest`. Update both in one pass.

ALGORITHM

- 1. Init $\text{largest} = -\infty$, $\text{second} = -\infty$.
- 2. For each element: if $\text{arr}[i] > \text{largest} \rightarrow \text{second} = \text{largest}$, $\text{largest} = \text{arr}[i]$.
- 3. Else if $\text{arr}[i] > \text{second}$ AND $\text{arr}[i] \neq \text{largest} \rightarrow \text{second} = \text{arr}[i]$.
- 4. Return second ($-\infty$ means no second largest exists).

EDGE CASE

- All elements equal $\rightarrow$ second largest doesn't exist. Check for $-\infty$ in result.

COMPLEXITY

Approach	Time	Space	Notes
Linear Scan	$O(N)$	$O(1)$	Single pass — optimal

■ Interview Trigger

Track two maxima simultaneously — update both in one scan.

3.1.3 · Check if Array is Sorted

TOPIC / PATTERN

Arrays — Linear Scan

ALGORITHM

- 1. For $i = 0$ to $N-2$: if $arr[i] > arr[i+1] \rightarrow$ return false.
- 2. Return true.

COMPLEXITY

Approach	Time	Space	Notes
Linear Scan	$O(N)$	$O(1)$	Stop at first violation

COMMON MISTAKES

- Checking $arr[i] \geq arr[i+1]$ — equal elements are fine in non-strictly sorted.

■ Interview Trigger

Check adjacent pairs — first violation means unsorted.

3.1.4 · Remove Duplicates from Sorted Array

TOPIC / PATTERN

Arrays — Two Pointers (slow/fast)

CORE IDEA

Use a slow pointer i to track the boundary of unique elements and a fast pointer j to scan forward.

ALGORITHM

- 1. $i = 0$ (last unique element position).
- 2. For $j = 1$ to $N-1$: if $arr[j] \neq arr[i]$, increment i , copy $arr[j]$ to $arr[i]$.
- 3. Return $i + 1$ (count of unique elements).

VISUALIZATION

```
[ 1, 1, 2, 3, 3, 4 ]
```

```
i → slow pointer (unique boundary)
```

```
j → fast scanner
```

```
arr[j] != arr[i] → i++, arr[i] = arr[j]
```

```
Result: [ 1, 2, 3, 4, _, _ ] return 4
```

COMPLEXITY

Approach	Time	Space	Notes
Two Pointers	$O(N)$	$O(1)$	In-place — optimal for sorted array

■ Interview Trigger

Slow pointer marks unique boundary — fast pointer discovers new values.

3.1.5 / 3.1.6 · Left Rotate Array by 1 and by D Places

TOPIC / PATTERN

Arrays — Reversal Algorithm

ROTATE BY 1

- Save $arr[0]$, shift all elements left by one, place saved at $arr[N-1]$.
- Time: $O(N)$ · Space: $O(1)$

ROTATE BY D — REVERSAL ALGORITHM (OPTIMAL)

- 1. Reverse the first D elements.
- 2. Reverse the remaining N-D elements.
- 3. Reverse the entire array.

VISUALIZATION (D=2, N=5)

```
[ 1, 2, 3, 4, 5 ] D=2
Step 1: Rev [0..1] → [ 2, 1, 3, 4, 5 ]
Step 2: Rev [2..4] → [ 2, 1, 5, 4, 3 ]
Step 3: Rev all → [ 3, 4, 5, 1, 2 ] ✓
```

COMPLEXITY

Approach	Time	Space	Notes
Extra Array	O(N)	O(N)	Simple but uses extra space
Reversal	O(N)	O(1)	Optimal — three reversal passes

COMMON MISTAKES

- Not handling $D > N$ — use $D = D \% N$ first.

■ Interview Trigger

Reverse first D, reverse rest, reverse all — rotation done in-place.

3.1.7 · Move Zeros to End**TOPIC / PATTERN**

Arrays — Two Pointers

CORE IDEA

Maintain a pointer j for the next non-zero position. Scan and swap non-zeros to j.

ALGORITHM

- 1. $j = 0$ (next position for non-zero element).
- 2. For $i = 0$ to $N-1$: if $arr[i] \neq 0$, swap $arr[i]$ and $arr[j]$, $j++$.

VISUALIZATION

```
[ 0, 1, 0, 3, 2 ]
j=0
i=1: arr[1]=1 ≠ 0 → swap(arr[1], arr[0]) → [ 1, 0, 0, 3, 2 ], j=1
i=3: arr[3]=3 ≠ 0 → swap(arr[3], arr[1]) → [ 1, 3, 0, 0, 2 ], j=2
i=4: arr[4]=2 ≠ 0 → swap(arr[4], arr[2]) → [ 1, 3, 2, 0, 0 ] ✓
```

COMPLEXITY

Approach	Time	Space	Notes
Two Pointers	O(N)	O(1)	Stable relative order of non-zeros maintained

■ Interview Trigger

Non-zero pointer tracks the front — push all non-zeros forward, zeros fall back.

3.1.10 · Find Missing Number in Array [0..N]**TOPIC / PATTERN**

Arrays — Math (Expected Sum) or XOR

APPROACH 1 — SUM FORMULA

- Expected sum of $0..N = N*(N+1)/2$.
- Missing = Expected Sum – Actual Sum.

APPROACH 2 — XOR (NO OVERFLOW RISK)

- XOR all array elements with XOR of $0..N$.
- Every present number cancels ($a \oplus a = 0$). Missing number remains.

COMPLEXITY

Approach	Time	Space	Notes
Sum Formula	$O(N)$	$O(1)$	Risk: integer overflow for large N
XOR	$O(N)$	$O(1)$	Preferred — no overflow possible

COMMON MISTAKES

- Sum approach: use long/int64 to avoid overflow.

■ Interview Trigger

XOR everything $0..N$ with all array elements — missing number survives.

3.1.11 · Maximum Consecutive Ones**TOPIC / PATTERN**

Arrays — Linear Scan with Counter

ALGORITHM

- 1. count = 0, maxCount = 0.
- 2. For each element: if 1 → count++, else count = 0.
- 3. maxCount = max(maxCount, count) every step.

COMPLEXITY

Approach	Time	Space	Notes
Linear Scan	$O(N)$	$O(1)$	Single pass

■ Interview Trigger

Reset counter on 0, track running max — one pass.

3.1.12 · Find the Single Number (Others Appear Twice)**TOPIC / PATTERN**

Arrays / Bit Manipulation — XOR

CORE IDEA

XOR of a number with itself = 0. XOR of a number with 0 = itself. XOR all elements — pairs cancel, the single number remains.

ALGORITHM

- result = 0. For each element: result ^= arr[i]. Return result.

VISUALIZATION

```
[ 4, 1, 2, 1, 2 ]
0 ^ 4 ^ 1 ^ 2 ^ 1 ^ 2
= 0 ^ 4 ^ (1^1) ^ (2^2)
= 0 ^ 4 ^ 0 ^ 0 = 4 ✓
```

COMPLEXITY

Approach	Time	Space	Notes
HashMap	$O(N)$	$O(N)$	Count frequencies — straightforward
XOR	$O(N)$	$O(1)$	Optimal — no extra space

■ Interview Trigger

XOR all elements — pairs vanish, the lone number survives.

3.1.13 / 3.1.14 · Longest Subarray with Sum K

TOPIC / PATTERN

Arrays — Sliding Window (positives only) or Prefix Sum + HashMap (pos+neg)

CASE 1 — ALL POSITIVE ELEMENTS: SLIDING WINDOW

- Expand window by moving right. If $\text{sum} > K$, shrink from left.
- Track max window size whenever $\text{sum} == K$.

CASE 2 — POSITIVES + NEGATIVES: PREFIX SUM + HASHMAP

- Maintain running prefix sum. Store first occurrence index of each prefix sum.
- If $(\text{prefixSum} - K)$ exists in map $\rightarrow$ subarray from that index+1 to current has sum K.
- Update max length accordingly.

VISUALIZATION — PREFIX SUM APPROACH

```
arr = [ 1, -1, 5, -2, 3 ], K = 3
```

```
prefix: 0, 1, 0, 5, 3, 6
```

```
At i=4: prefix=3, 3-3=0 exists at idx 0  $\rightarrow$  length = 4-0 = 4 ✓
```

COMPLEXITY

Approach	Time	Space	Notes
Brute	$O(N^2)$	$O(1)$	Check all subarrays — TLE for large N
Sliding Window	$O(N)$	$O(1)$	Only valid for non-negative arrays
Prefix+HashMap	$O(N)$	$O(N)$	Works for all cases including negatives

COMMON MISTAKES

- Using sliding window on arrays with negative numbers — window won't shrink correctly.
- Not initializing map with $\{0: -1\}$ — misses subarrays starting from index 0.

■ Interview Trigger

Positives only $\rightarrow$ sliding window. Negatives present $\rightarrow$ prefix sum + hashmap.

3.2 MEDIUM PROBLEMS

3.2.1 · Two Sum

TOPIC / PATTERN

Arrays — HashMap or Two Pointers

CORE IDEA

For each element x , check if $(\text{target} - x)$ has been seen before. If yes, we have our pair.

ALGORITHM — HASHMAP

- 1. For each index i : $\text{complement} = \text{target} - \text{arr}[i]$.
- 2. If complement exists in map $\rightarrow$ return $[\text{map}[\text{complement}], i]$.
- 3. Otherwise store $\text{arr}[i] \rightarrow i$ in the map.

ALGORITHM — TWO POINTERS (SORTED ARRAY ONLY)

- 1. Sort the array (or use original indices carefully).
- 2. $L = 0, R = N-1$. While $L < R$:
- 3. $\text{sum} = \text{arr}[L] + \text{arr}[R]$. If $\text{sum} == \text{target} \rightarrow$ found. If $\text{sum} < \text{target} \rightarrow L++$. Else $R--$.

COMPLEXITY

Approach	Time	Space	Notes
Brute	$O(N^2)$	$O(1)$	Check all pairs
HashMap	$O(N)$	$O(N)$	One pass — handles unsorted + returns indices
Two Pointers	$O(N \log N)$	$O(1)$	Needs sorted array — lose original indices

INTERVIEW NOTE

- If the problem asks for indices $\rightarrow$ use HashMap (two pointers loses original indices after sort).
- If it asks for existence/values and array is sorted $\rightarrow$ use two pointers.

Related: [LeetCode 167 Two Sum II](#), [LeetCode 15 3Sum](#), [LeetCode 18 4Sum](#)

■ Interview Trigger

Store complement while scanning — $O(N)$ lookup via hashmap.

3.2.2 · Sort Array of 0s, 1s, 2s — Dutch National Flag

TOPIC / PATTERN

Arrays — Dutch National Flag (3-way partition with 3 pointers)

CORE IDEA

Maintain three regions: $[0..\text{low}-1] = 0\text{s}$, $[\text{low}..\text{mid}-1] = 1\text{s}$, $[\text{high}+1..N-1] = 2\text{s}$. mid is the explorer.

ALGORITHM

- 1. $\text{low} = 0, \text{mid} = 0, \text{high} = N-1$.
- 2. While $\text{mid} \leq \text{high}$:
- 3. $\text{arr}[\text{mid}] == 0 \rightarrow \text{swap}(\text{mid}, \text{low}), \text{low}++, \text{mid}++$.
- 4. $\text{arr}[\text{mid}] == 1 \rightarrow \text{mid}++$ (already in correct region).
- 5. $\text{arr}[\text{mid}] == 2 \rightarrow \text{swap}(\text{mid}, \text{high}), \text{high}--$ (DO NOT $\text{mid}++$ — check new $\text{arr}[\text{mid}]$).

VISUALIZATION

```
[ 2, 0, 1, 2, 0, 1 ]
```

```
L H
```

```
M
```



```
arr[M]=2 → swap(M,H), H-- → [ 1, 0, 1, 2, 0, 2 ]
arr[M]=1 → M++ → [ 1, 0, 1, 2, 0, 2 ]
... continues until [ 0, 0, 1, 1, 2, 2 ] ✓
```

COMPLEXITY

Approach	Time	Space	Notes
Count & Fill	O(N)	O(1)	Two passes — count 0s/1s/2s, then overwrite
Dutch National Flag	O(N)	O(1)	One pass — optimal

COMMON MISTAKES

- Incrementing mid after swapping with high — the swapped-in element is unchecked.

Related: [LeetCode 75 Sort Colors](#)

■ Interview Trigger

Three pointers partition in one pass — never move mid after swapping with high.

3.2.3 · Majority Element (> N/2 times) — Boyer-Moore Voting**TOPIC / PATTERN**

Arrays — Boyer-Moore Voting Algorithm

CORE IDEA

The majority element (appearing > N/2 times) can never be fully cancelled out. Use a candidate and a vote count — votes cancel between different elements.

ALGORITHM

1. candidate = arr[0], votes = 1.
2. For i = 1 to N-1: if votes == 0 → candidate = arr[i], votes = 1.
3. Else if arr[i] == candidate → votes++. Else votes--.
4. Verify candidate appears > N/2 times (if problem guarantees majority exists, skip verification).

VISUALIZATION

```
[ 2, 2, 1, 1, 2 ]
cand=2 votes=1
arr[1]=2 → votes=2
arr[2]=1 → votes=1
arr[3]=1 → votes=0
arr[4]=2 → cand=2, votes=1 → candidate = 2 ✓
```

COMPLEXITY

Approach	Time	Space	Notes
Brute	O(N ²)	O(1)	Count each element explicitly
HashMap	O(N)	O(N)	Store frequencies
Sorting	O(N log N)	O(1)	N/2 index always holds majority
Boyer-Moore	O(N)	O(1)	Optimal — single pass, no extra space

COMMON MISTAKES

- Not verifying the candidate — if majority not guaranteed to exist, must verify.

Related: [LeetCode 169 Majority Element](#), [LeetCode 229 Majority Element II](#)

■ Interview Trigger

Majority element can never be voted out — last standing candidate wins.

3.2.4 / 3.2.5 · Kadane's Algorithm — Maximum Subarray Sum ■

TOPIC / PATTERN

Arrays — Kadane's Algorithm (Dynamic Programming / Greedy)

CORE IDEA

At each index, the maximum subarray ending here is either: (a) just the current element (start fresh), or (b) extend the previous subarray. Carry the sum forward only if it's positive.

ALGORITHM

- 1. $\text{maxSum} = \text{arr}[0]$, $\text{currentSum} = \text{arr}[0]$.
- 2. For $i = 1$ to $N-1$:
- 3. $\text{currentSum} = \max(\text{arr}[i], \text{currentSum} + \text{arr}[i])$.
- 4. $\text{maxSum} = \max(\text{maxSum}, \text{currentSum})$.
- 5. Return maxSum .

EXTENDED — PRINT THE SUBARRAY

- Track start and end indices. When $\text{currentSum} < \text{arr}[i]$, reset $\text{start} = i$.
- When maxSum updates, record $\text{end} = i$ and save start.

VISUALIZATION

```
[ -2, 1, -3, 4, -1, 2, 1, -5, 4 ]
cur: -2 1 -2 4 3 5 6 1 5
max: -2 1 1 4 4 5 6 6 6 → maxSum = 6
Subarray: [ 4, -1, 2, 1 ] ✓
```

KEY INSIGHT

- If currentSum goes negative, adding it to a future element only hurts — reset to 0.

COMPLEXITY

Approach	Time	Space	Notes
Brute	$O(N^3)$	$O(1)$	All subarrays + sum each
Better	$O(N^2)$	$O(1)$	Prefix sum to compute subarray sum in $O(1)$
Kadane's	$O(N)$	$O(1)$	Optimal — single pass

COMMON MISTAKES

- Initializing $\text{maxSum} = 0$ — fails for all-negative arrays. Use $\text{arr}[0]$ or $-\infty$.

Related: [LeetCode 53 Maximum Subarray](#), [LeetCode 152 Maximum Product Subarray](#)

■ Interview Trigger

A negative running sum only hurts — reset to 0 and start fresh.

3.2.6 · Stock Buy and Sell (Single Transaction)

TOPIC / PATTERN

Arrays — Greedy / Track Running Minimum

CORE IDEA

For each day, compute profit if we sell today ($\text{price}[i] - \text{min_price_so_far}$). Track the running minimum buy price and the maximum profit seen.

ALGORITHM

- 1. minPrice = prices[0], maxProfit = 0.
- 2. For i = 1 to N-1:
- 3. profit = prices[i] – minPrice.
- 4. maxProfit = max(maxProfit, profit).
- 5. minPrice = min(minPrice, prices[i]).

COMPLEXITY

Approach	Time	Space	Notes
Brute	O(N ²)	O(1)	Try all buy-sell pairs
Optimal	O(N)	O(1)	Track running min — single pass

COMMON MISTAKES

- Selling before buying — always update minPrice AFTER computing profit at i.

Related: LeetCode 121, 122 (multiple), 123 (at most 2), 188 (at most K) — all DP variants

■ Interview Trigger

Track the cheapest buy price so far — profit at each day = price – min_so_far.

3.2.8 · Next Permutation**TOPIC / PATTERN**

Arrays — 3-Step Algorithm (Find breakpoint, find swap, reverse)

CORE IDEA

Find the longest decreasing suffix — just before it is the 'breakpoint'. Swap breakpoint with the next greater element in the suffix, then reverse the suffix.

ALGORITHM

- 1. Find the rightmost index i where arr[i] < arr[i+1] — this is the breakpoint.
- 2. If no such index exists → array is the last permutation → reverse entire array.
- 3. Find the rightmost j > i where arr[j] > arr[i]. Swap arr[i] and arr[j].
- 4. Reverse the suffix arr[i+1 .. N-1].

VISUALIZATION

```
[ 1, 3, 5, 4, 2 ]
Step 1: breakpoint i=1 (arr[1]=3 < arr[2]=5)
Step 3: rightmost j where arr[j] > 3 → j=3 (arr[3]=4)
swap → [ 1, 4, 5, 3, 2 ]
Step 4: reverse suffix [2..4] → [ 1, 4, 2, 3, 5 ] ✓
```

COMPLEXITY

Approach	Time	Space	Notes
3-Step	O(N)	O(1)	In-place — optimal

COMMON MISTAKES

- Reversing instead of sorting the suffix — since it's already in descending order, reversing gives ascending.

Related: LeetCode 31 Next Permutation, LeetCode 46/47 Permutations

■ Interview Trigger

Find dip → swap with next-greater → reverse suffix → next permutation found.

3.2.10 · Longest Consecutive Sequence

TOPIC / PATTERN

Arrays — HashSet

CORE IDEA

For each number that is the START of a sequence (num-1 not in set), count how long the streak extends. Only start counting from sequence starts.

ALGORITHM

- 1. Insert all elements into a HashSet.
- 2. For each num: if (num-1) not in set → num is a sequence start.
- 3. Count streak: while (num + streak) in set → streak++.
- 4. Update maxStreak.

COMPLEXITY

Approach	Time	Space	Notes
Brute	$O(N^3)$	$O(1)$	Check all pairs
Sorting	$O(N \log N)$	$O(1)$	Sort and count consecutive runs
HashSet	$O(N)$	$O(N)$	Optimal — each element processed at most twice

COMMON MISTAKES

- Not checking if num-1 exists before counting — leads to $O(N^2)$ from redundant counts.

Related: [LeetCode 128 Longest Consecutive Sequence](#)

■ Interview Trigger

Only count from sequence starts (num-1 absent) — each number touched at most twice.

3.2.11 · Set Matrix Zeros

TOPIC / PATTERN

Arrays / Matrix — In-Place Marking

CORE IDEA

Use the first row and first column as markers to avoid extra space. A separate flag tracks whether the first row/column themselves should be zeroed.

ALGORITHM

- 1. Check if first row or first column contains 0 (save flags firstRowZero, firstColZero).
- 2. For all (i, j) with $i > 0, j > 0$: if $\text{matrix}[i][j] == 0$, mark $\text{matrix}[i][0] = 0$ and $\text{matrix}[0][j] = 0$.
- 3. For $i > 0, j > 0$: if $\text{matrix}[i][0] == 0$ or $\text{matrix}[0][j] == 0$ → set $\text{matrix}[i][j] = 0$.
- 4. Use flags to zero out first row/column if needed.

COMPLEXITY

Approach	Time	Space	Notes
Extra Space	$O(M \times N)$	$O(M+N)$	Store zero positions separately
Row/Col Sets	$O(M \times N)$	$O(M+N)$	Track rows and cols with zeros
In-Place	$O(M \times N)$	$O(1)$	Use first row/col as markers — optimal

COMMON MISTAKES

- Zeroing cells immediately while scanning — corrupts markers. Always do it in two passes.

Related: [LeetCode 73 Set Matrix Zeroes](#)

■ Interview Trigger

Use first row and column as markers — two-pass in-place $O(1)$ space.

3.2.12 · Rotate Matrix 90 Degrees Clockwise

TOPIC / PATTERN

Matrix — Transpose + Reverse Rows

CORE IDEA

90° clockwise rotation = Transpose the matrix, then reverse each row.

ALGORITHM

- 1. Transpose: swap $\text{matrix}[i][j]$ with $\text{matrix}[j][i]$ for all $i < j$.
- 2. Reverse each row: reverse the elements within every row.

VISUALIZATION

Original: Transposed: Reversed rows:

1 2 3 1 4 7 7 4 1

4 5 6 → 2 5 8 → 8 5 2

7 8 9 3 6 9 9 6 3 ✓

ANTI-CLOCKWISE

- Transpose first, then reverse each column (or reverse rows first, then transpose).

COMPLEXITY

Approach	Time	Space	Notes
Extra Matrix	$O(N^2)$	$O(N^2)$	Direct copy to new matrix
In-Place	$O(N^2)$	$O(1)$	Transpose + reverse — optimal

Related: [LeetCode 48 Rotate Image](#)

■ Interview Trigger

Clockwise 90° = Transpose then reverse each row. Anti-clockwise = reverse each row then transpose.

3.2.13 · Print Matrix in Spiral Order

TOPIC / PATTERN

Matrix — Boundary Shrinking with Four Pointers

ALGORITHM

- 1. Define boundaries: $\text{top}=0$, $\text{bottom}=\text{M}-1$, $\text{left}=0$, $\text{right}=\text{N}-1$.
- 2. While $\text{top} \leq \text{bottom}$ and $\text{left} \leq \text{right}$:
- 3. Print $\text{left} \rightarrow \text{right}$ along top row. $\text{top}++$.
- 4. Print $\text{top} \rightarrow \text{bottom}$ along right column. $\text{right}--$.
- 5. If $\text{top} \leq \text{bottom}$: print $\text{right} \rightarrow \text{left}$ along bottom row. $\text{bottom}--$.
- 6. If $\text{left} \leq \text{right}$: print $\text{bottom} \rightarrow \text{top}$ along left column. $\text{left}++$.

COMPLEXITY

Approach	Time	Space	Notes
Four Pointers	$O(M \times N)$	$O(1)$	Visit each cell exactly once

COMMON MISTAKES

- Not checking $\text{top} \leq \text{bottom}$ / $\text{left} \leq \text{right}$ before printing bottom row / left column — causes duplicates in single-row/column matrices.

Related: [LeetCode 54 Spiral Matrix](#), [LeetCode 59 Spiral Matrix II](#)

■ Interview Trigger

Shrink four boundaries layer by layer — always check bounds before each direction.

3.2.14 · Count Subarrays with Given Sum K

TOPIC / PATTERN

Arrays — Prefix Sum + HashMap

CORE IDEA

A subarray $[j+1..i]$ has sum K when $\text{prefixSum}[i] - \text{prefixSum}[j] == K$, i.e. $\text{prefixSum}[j] == \text{prefixSum}[i] - K$. Count how many times that value appeared.

ALGORITHM

- 1. $\text{prefixSum} = 0$, $\text{count} = 0$. $\text{Map} = \{0: 1\}$ (empty prefix).
- 2. For each element: $\text{prefixSum} += \text{arr}[i]$.
- 3. $\text{count} += \text{map.get}(\text{prefixSum} - K, 0)$.
- 4. $\text{map}[\text{prefixSum}]++$.

COMPLEXITY

Approach	Time	Space	Notes
Brute	$O(N^2)$	$O(1)$	All pairs
Prefix+HashMap	$O(N)$	$O(N)$	Optimal

COMMON MISTAKES

- Forgetting to initialize map with $\{0: 1\}$ — misses subarrays that start from index 0.
- Querying map BEFORE updating it — lookup first, then insert.

Related: [LeetCode 560 Subarray Sum Equals K](#)

■ Interview Trigger

Prefix sum magic: count how many times $(\text{prefixSum} - K)$ appeared before.

3.3 HARD PROBLEMS

3.3.1 · Pascal's Triangle

TOPIC / PATTERN

Arrays / Math — Three Variants

VARIANT 1 — VALUE AT (ROW, COL)

- Answer = $C(\text{row}-1, \text{col}-1) = (\text{row}-1)! / ((\text{col}-1)! \times (\text{row}-\text{col})!)$.
- Compute iteratively: multiply $(\text{row}-1)(\text{row}-2)\dots(\text{row}-\text{col}+1) / (\text{col}-1)!$

VARIANT 2 — PRINT ROW R

- Start with 1. Multiply by $(R-1-i) / (i+1)$ for $i = 0$ to $R-2$.

VARIANT 3 — PRINT ALL N ROWS

- Each row i : start with [1]. Each element = sum of two elements directly above.

VISUALIZATION

```
Row 1: 1
Row 2: 1 1
Row 3: 1 2 1
Row 4: 1 3 3 1
Row 5: 1 4 6 4 1
```

COMPLEXITY

Approach	Time	Space	Notes
Optimal	$O(N^2)$	$O(N^2)$	Build row by row

■ Interview Trigger

Pascal's cell = $C(\text{row}-1, \text{col}-1)$. Each row element = sum of two above.

3.3.2 · Majority Element (> N/3 times) — Extended Voting

TOPIC / PATTERN

Arrays — Extended Boyer-Moore Voting (2 candidates)

CORE IDEA

At most 2 elements can appear $> N/3$ times. Run voting with 2 candidate slots — eliminate triplets of distinct elements.

ALGORITHM

1. Find candidates: two slots (cand1, votes1), (cand2, votes2).
2. For each element: if it matches cand1 $\rightarrow$ votes1++. Else if matches cand2 $\rightarrow$ votes2++.
3. Else if votes1==0 $\rightarrow$ cand1=element, votes1=1. Else if votes2==0 $\rightarrow$ cand2=element, votes2=1.
4. Else votes1--, votes2--.
5. Verify both candidates by counting actual occurrences — keep those with count $> N/3$.

COMPLEXITY

Approach	Time	Space	Notes
HashMap	$O(N)$	$O(N)$	Count frequencies
Extended Vote	$O(N)$	$O(1)$	Optimal — two-pass

COMMON MISTAKES

- Not verifying candidates at the end — voting only finds candidates, not guarantees.

Related: [LeetCode 229 Majority Element II](#)

■ Interview Trigger

At most 2 majority ($>N/3$) elements exist — use 2 candidate slots and verify.

3.3.3 / 3.3.4 · 3-Sum and 4-Sum Problems

TOPIC / PATTERN

Arrays — Sort + Two Pointers (avoid duplicates carefully)

3-SUM ALGORITHM

- 1. Sort the array.
- 2. Fix element $arr[i]$. Use two pointers $L=i+1$, $R=N-1$ to find pair summing to $-arr[i]$.
- 3. If $sum < target \rightarrow L++$. If $sum > target \rightarrow R--$. If $sum == target \rightarrow$ record, skip duplicates.
- 4. Skip duplicate values of $arr[i]$ to avoid duplicate triplets.

4-SUM ALGORITHM

- Add an outer loop fixing $arr[j]$, then run 3-Sum logic inside. $O(N^3)$ total.

COMPLEXITY

Approach	Time	Space	Notes
Brute 3Sum	$O(N^3)$	$O(1)$	All triplets — TLE
Sort+Two Pointers	$O(N^2)$	$O(1)$	Optimal for 3Sum
4Sum	$O(N^3)$	$O(1)$	Two fixed + two pointer

COMMON MISTAKES

- Not skipping duplicates $\rightarrow$ duplicate triplets/quadruplets in output.
- Integer overflow in 4Sum — use long for the target comparisons.

Related: [LeetCode 15 3Sum](#), [LeetCode 18 4Sum](#), [LeetCode 16 3Sum Closest](#)

■ Interview Trigger

Sort first, fix outer elements, use two pointers inside — skip duplicates at every level.

3.3.5 · Largest Subarray with 0 Sum

TOPIC / PATTERN

Arrays — Prefix Sum + HashMap (store first occurrence index)

CORE IDEA

If the prefix sum at index i equals the prefix sum at index j ($i < j$), then the subarray $(i+1, j)$ has sum 0. Find the largest such span.

ALGORITHM

- 1. $Map = \{0: -1\}$ (prefix sum 0 at index -1 before the array).
- 2. $prefixSum = 0$. For each i :
- 3. $prefixSum += arr[i]$.
- 4. If $prefixSum$ in $map \rightarrow length = i - map[prefixSum]$. Update $maxLen$.
- 5. Else $map[prefixSum] = i$ (store FIRST occurrence only).

COMPLEXITY

Approach	Time	Space	Notes
Brute	$O(N^2)$	$O(1)$	All subarrays
Prefix+HashMap	$O(N)$	$O(N)$	Optimal

COMMON MISTAKES

- Overwriting map with later indices — we want the earliest occurrence for maximum length.

■ Interview Trigger

Same prefix sum twice means zero-sum subarray in between — maximize the gap.

3.3.6 · Count Subarrays with XOR = K**TOPIC / PATTERN**

Arrays — Prefix XOR + HashMap

CORE IDEA

Analogous to prefix sum. If $\text{prefixXOR}[i] \text{ XOR } \text{prefixXOR}[j] == K$, then subarray $(j+1, i)$ has XOR = K. Equivalently: $\text{prefixXOR}[j] == \text{prefixXOR}[i] \text{ XOR } K$.

ALGORITHM

- 1. Map = {0: 1}. prefixXOR = 0, count = 0.
- 2. For each element: $\text{prefixXOR} \wedge= \text{arr}[i]$.
- 3. $\text{count} += \text{map.get}(\text{prefixXOR} \wedge K, 0)$.
- 4. $\text{map}[\text{prefixXOR}]++$.

COMPLEXITY

Approach	Time	Space	Notes
Brute	$O(N^2)$	$O(1)$	All subarrays
Prefix XOR Map	$O(N)$	$O(N)$	Optimal

KEY OBSERVATION

- XOR obeys the same prefix identity as sum: if $A \text{ XOR } B = K$, then $A = B \text{ XOR } K$.

■ Interview Trigger

Replace 'sum' with 'XOR' — prefix XOR map works identically to prefix sum map.

3.3.7 · Merge Overlapping Intervals**TOPIC / PATTERN**

Arrays — Sort by Start + Greedy Merge

CORE IDEA

Sort intervals by start time. Scan linearly — if the current interval overlaps the last merged interval, extend it. Otherwise start a new merged interval.

ALGORITHM

- 1. Sort intervals by start time.
- 2. Add first interval to result.
- 3. For each subsequent interval:
- 4. If $\text{start} \leq \text{result.last.end} \rightarrow \text{overlap} \rightarrow \text{update result.last.end} = \max(\text{result.last.end}, \text{end})$.
- 5. Else $\rightarrow$ no overlap $\rightarrow$ append interval to result.

VISUALIZATION

```

Intervals: [(1,3), (2,6), (8,10), (15,18)]
Sorted: [(1,3), (2,6), (8,10), (15,18)]
Merge (1,3)+(2,6) → (1,6)
(1,6) and (8,10) → no overlap → add (8,10)
Result: [(1,6), (8,10), (15,18)] ✓

```

COMPLEXITY

Approach	Time	Space	Notes
Sort+Scan	$O(N \log N)$	$O(N)$	Dominated by sort

COMMON MISTAKES

- Forgetting to sort first — algorithm only works on sorted intervals.

Related: [LeetCode 56 Merge Intervals](#), [LeetCode 57 Insert Interval](#)

■ Interview Trigger

Sort by start → scan and greedily merge overlapping intervals.

3.3.8 · Merge Two Sorted Arrays Without Extra Space**TOPIC / PATTERN**

Arrays — Gap Method (Shell Sort variant) or Two Pointer with Swap

APPROACH 1 — TWO POINTER + SWAP (SIMPLE)

- i = end of arr1, j = start of arr2. While $i \geq 0$ and $j < \text{len}(\text{arr2})$:
- If $\text{arr1}[i] > \text{arr2}[j] \rightarrow$ swap them. Then sort each array. $O((M+N) \log(M+N))$ effectively.

APPROACH 2 — GAP METHOD (OPTIMAL $O(1)$ SPACE)

- $\text{gap} = \text{ceil}((M+N)/2)$. Compare elements that are 'gap' apart across both arrays and swap if needed.
- Halve gap each iteration. Repeat until $\text{gap} = 0$.

COMPLEXITY

Approach	Time	Space	Notes
Swap+Sort	$O((M+N) \log(M+N))$	$O(1)$	Simpler to implement
Gap Method	$O((M+N) \log(M+N))$	$O(1)$	True $O(1)$ — gap halves each pass

COMMON MISTAKES

- Using extra space — question specifically forbids it.

Related: [LeetCode 88 Merge Sorted Array \(allows extra space\)](#)

■ Interview Trigger

Gap method: compare elements gap-positions apart, swap if needed, halve gap — $O(1)$ space.

3.3.9 · Find Repeating and Missing Number**TOPIC / PATTERN**

Arrays — Math (Sum + Sum of Squares) or XOR

CORE IDEA — MATH APPROACH

- Let repeating = A, missing = B.
- $S1 = \text{sum}(\text{arr}) - \text{expected_sum} \rightarrow A - B = S1$.
- $S2 = \text{sum}(\text{arr}^2) - \text{expected_sum_squares} \rightarrow A^2 - B^2 = S2 \rightarrow (A+B) = S2/S1$.
- Solve: $A = (S1 + S2/S1) / 2$, $B = A - S1$.

COMPLEXITY

Approach	Time	Space	Notes
Brute	$O(N^2)$	$O(1)$	Count each number
HashMap	$O(N)$	$O(N)$	Store frequencies
Math	$O(N)$	$O(1)$	Two equations — optimal
XOR	$O(N)$	$O(1)$	Bit-wise — no overflow

COMMON MISTAKES

- Integer overflow in sum of squares — use long/int64.

■ Interview Trigger

Two equations, two unknowns: $(A-B)$ from sum, $(A+B)$ from sum of squares.

3.3.10 / 3.3.11 · Count Inversions and Reverse Pairs**TOPIC / PATTERN**

Arrays — Modified Merge Sort

CORE IDEA

During the merge step of merge sort, when a right-half element is placed before left-half elements, all remaining left-half elements form inversions (or reverse pairs) with it. Count them.

COUNT INVERSIONS — $(I < J \text{ AND } \text{ARR}[I] > \text{ARR}[J])$

- During merge: if $\text{arr}[\text{left}] > \text{arr}[\text{right}]$, all remaining left-half elements ($\text{mid} - \text{left} + 1$) form inversions with $\text{arr}[\text{right}]$.

REVERSE PAIRS — $(I < J \text{ AND } \text{ARR}[I] > 2 \times \text{ARR}[J])$

- Before merging, use two pointers to count pairs satisfying $\text{arr}[i] > 2 \times \text{arr}[j]$ across halves.
- Then perform the regular merge to sort.

COMPLEXITY

Approach	Time	Space	Notes
Brute	$O(N^2)$	$O(1)$	All pairs — TLE
Merge Sort	$O(N \log N)$	$O(N)$	Count during merge — optimal

COMMON MISTAKES

- For reverse pairs: counting and merging in the same pass gives wrong results — count first, then merge.

Related: [LeetCode 493 Reverse Pairs](#), [LeetCode 315 Count Smaller After Self](#)

■ Interview Trigger

Embed the count inside merge sort — inversions accumulate during the merge step.

3.3.12 · Maximum Product Subarray**TOPIC / PATTERN**

Arrays — Track Both Max and Min (negatives flip signs)

CORE IDEA

A negative number can turn the minimum product into the maximum. Track both the current max and current min product at every step.

ALGORITHM

- 1. $\text{maxProd} = \text{arr}[0]$, $\text{minProd} = \text{arr}[0]$, $\text{result} = \text{arr}[0]$.
- 2. For $i = 1$ to $N-1$:
- 3. $\text{candidates} = \{\text{arr}[i], \text{arr}[i] \times \text{maxProd}, \text{arr}[i] \times \text{minProd}\}$.
- 4. $\text{maxProd} = \max(\text{candidates})$, $\text{minProd} = \min(\text{candidates})$.
- 5. $\text{result} = \max(\text{result}, \text{maxProd})$.

WHY TRACK MINIMUM?

[2, 3, -2, 4]

At -2: $\text{maxProd}=6$, $\text{minProd}=6$

$\text{new max} = \max(-2, 6 \times -2, 6 \times -2) = -2$

$\text{new min} = \min(-2, -12, -12) = -12$

At 4: $\text{max} = \max(4, -8, -48) = 4 \rightarrow \text{result} = 6$

COMPLEXITY

Approach	Time	Space	Notes
Brute	$O(N^2)$	$O(1)$	All subarrays
Optimal	$O(N)$	$O(1)$	Track both max and min

COMMON MISTAKES

- Only tracking max — misses the negative×negative opportunity.
- Forgetting 0 resets both max and min to 0 (handled automatically by taking $\text{arr}[i]$ alone as candidate).

Related: [LeetCode 152 Maximum Product Subarray](#), [LeetCode 53 Maximum Subarray \(sum\)](#)

■ Interview Trigger

Track both max and min — a negative flips them, unlocking hidden maximums.